Response to Reviewer 3

Enhancing Consent Policy Privacy in Blockchain-Based Healthcare Data Sharing

---

Dear Reviewer 3,

We thank you for your review and comments, which have helped us to improve the manuscript considerably. Below we provide a point-by-point response to your comments, and we have revised the manuscript accordingly.

Comments 1: *The trust model around the Trusted Proof Service is a major limitation and should be discussed more critically. The paper explicitly assumes that the TPS preserves policy material and requester evidence, but the current evaluation even co-locates the TPS with the provider. This weakens the practical privacy claim, since the most sensitive information is moved off-chain rather than cryptographically protected from the service itself. The paper should more clearly distinguish what is protected from public-chain observers and what is merely trusted to the TPS.*

Response 1: We agree that this is the main architectural limitation, and the revision now states it as such. Section 4.1 separates what is protected cryptographically from what is only trusted to the TPS. Authorization correctness is protected: a compromised TPS cannot alter the program length, the authorization context, or the signed claims. Policy confidentiality and liveness are only trusted to the TPS, and the provider-operated TPS in our evaluation concentrates this trust at the provider. The new Table 3 lists these guarantees for each actor and failure mode. The Discussion and Conclusion describe confidentiality against the TPS as a trust assumption, and the Discussion names mechanisms that would reduce it.

Revised in: Section 4.1, third and fourth paragraphs (page 8, lines 288–310); Table 3 (page 9); Section 7, paragraphs "TPS trust and credential provenance" (page 33, lines 1147–1153) and "Future directions" (page 34, lines 1198–1205); Section 8, last sentence (page 34, lines 1220–1222).

Comments 2: *The current proof statement leaves important soundness gaps. The paper acknowledges that the program length is not bound into the policy commitment and that the proof is not bound to the requester, resource, or session context. These are not minor implementation details: they directly affect the correctness and replay-resistance of the authorization workflow. These issues should be fixed in the design and experimentally re-evaluated, rather than mainly left to future work.*

Response 2: Both gaps are now closed in the design and the implementation, and the evaluation was repeated on the revised system. The policy commitment binds the program length (88 additional constraints, no public input). The proof carries a context digest over requester, resource, and nonce, which the contract recomputes. This digest adds 605 constraints, one public input, and 64,022 gas per access, and a replayed proof with fresh parameters is rejected. All results in the revised manuscript come from a new run of 1,077 policies and 1,773 Polygon Amoy executions, plus a new 1,119-execution unpadded ablation. The cost profile reflects the revised circuit (81,740 constraints).

Revised in: Section 4.4, first requirement (pages 10–11, lines 384–388); Section 5.3 (pages 13–14, lines 504–518; pages 14–15, lines 519–555); Section 6.3, paragraph "Shared corpus" (page 20, lines 763–770); Tables 6 (page 17) and 15 (page 31); all result tables of Section 6.

Comments 3: *The policy language is still quite limited. The system focuses on fixed-length Boolean RPN programs, while many realistic healthcare consent policies involve temporal constraints, revocation, obligations, delegation, purpose limitation, institutional approval, and evolving consent. Some of these are treated as external predicate bits, but this moves complexity outside the proven statement. The paper should better explain what kinds of real consent policies can actually be supported end-to-end.*

Response 3: We now state exactly which consent policies are supported end-to-end. Every predicate is now derived in-circuit from evidence bound to committed parameters rather than supplied as an unconstrained bit. Ontology conditions use Merkle membership proofs; user-, project-, and institution-specific restrictions use allowlist membership; and profile-backed terms use signed credential claims. The policy validity window is checked against the public request epoch, and the contract enforces time-limited use from public state. A new table classifies each consent feature as checked in-circuit, checked on-chain, committed but not decided at request time, or not supported. Institutional or ethics approval, downstream purpose limitation, post-access obligations, delegation, and credential revocation remain outside the proved statement, and Section 5.2 states this explicitly.

Revised in: Section 5.2, paragraph "Scope of the supported policies" (page 13, lines 483–502); Table 4 (page 14); Section 7, second and third paragraphs under "Issuer governance and deployment path" (pages 33–34, lines 1168–1183).

Comments 4: *The privacy evaluation is too narrow. The adversary is mainly a passive metadata-only public-chain observer, and the effective leakage channel is reduced mostly to gas usage. This is useful, but it does not cover adaptive requester probing, repeated access attempts, timing/order leakage, off-chain TPS communication patterns, compromised TPS behaviour, or correlations between authorization outcomes and policy classes. The paper should either broaden the evaluation or tone down the privacy claims.*

Response 4: We agree your comment. The Abstract, Discussion, and Conclusion now claim only empirical resistance against the modeled passive metadata adversary, and Section 6.1 lists each channel you named as excluded. In the re-run evaluation, two of the 45 tasks fail; we report them, trace them to the random padding targets, and quantify the criterion's false-positive rate. The re-run ablation also identified the source of the unpadded signal, the calldata byte pricing of the requester pseudonym, which shows that the evaluation detects small deterministic channels.

Revised in: Abstract (page 1, lines 2–17); Section 6.1, third paragraph (pages 17–18, lines 619–631); Section 6.9, paragraph "Source of the unpadded signal" and the two paragraphs that follow (pages 29–30, lines 1030–1054); Section 7, paragraphs under "Privacy findings" (page 32, lines 1101–1123); Section 8, last sentence (page 34, lines 1220–1222).

Comments 5: *The comparison with related work should be strengthened by considering recent blockchain-based access-control architectures that combine smart contracts, SSI-oriented trust models, and cryptographic mechanisms for controlling access to sensitive claims. In particular, the paper should better position TPS-ZKPI with respect to the issues discussed in DOI: 10.1016/j.bcra.2024.100196, especially regarding verifier authorization, smart-contract accountability, off-chain trust assumptions, and practical access-control workflows.*

Response 5: We added a paragraph to the Related Work section and a row to Table 2 that position TPS-ZKPI against Buccafurri et al. [27] on the four points raised (A–D below).

(A) *Verifier authorization:* their design releases the holder's claims only to verifiers whose certified attributes satisfy a holder-defined policy; in TPS-ZKPI the protected object is the

provider's consent policy, and the verifier is a public contract that any observer can check.

(B) *Smart-contract accountability:* their contract notarizes hashed operations for later attribution, whereas ours verifies the proof and records the context-bound outcome on-chain.

(C) *Off-chain trust:* their proxy is semi-trusted and never sees claim contents, whereas our TPS sees both the policy and the claims; the paragraph states this difference and refers to Table 3.

(D) *Workflow:* the designs are complementary, since a requester could use their mechanism to release claims only to an accredited TPS.

We also cite recent zero-knowledge authentication work [28] and a survey [29].

Revised in: Section 3, fourth paragraph (page 6, lines 210–232); Table 2 (page 7); References.

Comments 6: *The cost and scalability analysis needs more depth. The reported gas-padding overhead is substantial, around 38% for access and 40% for commit operations, and the evaluation is performed at a single circuit depth. The paper should evaluate how costs change with larger policy programs, richer predicates, more complex DUO/ICD conditions, and more realistic access workloads. Otherwise, it is difficult to judge whether the design is practical for real healthcare data-sharing platforms.*

Response 6: We added offline scalability measurements and a workload analysis. Increasing the program length from 32 to 128 raises the circuit from 64,945 to 134,009 constraints and the median proving time from 0.41 to 0.73 s; increasing the ontology depth from 4 to 16 raises it from 0.51 to 0.57 s. Within the deployed circuit, policies of 3 to 61 tokens using one to all 11 predicate slots keep the median proving time at 0.48–0.55 s. All variants keep five public signals, so the verifier interface and the padding band are unchanged. For workloads, we measured a throughput of 0.936 proofs/s on one host, about 80,900 requests per day per prover; the Discussion combines this with the per-access gas cost, which grows linearly with the number of requests.

Revised in: Section 6.9, last two paragraphs (page 31, lines 1083–1099); Tables 16 and 17 (page 31); Section 7, paragraphs under "The cost of padding" (pages 32–33, lines 1124–1146).